

多元时间序列数据视角下高校就业指标分析与预测

摘要

本文针对高校毕业生就业率与时间的相关性问题，旨在建立一个能够准确描述并预测其动态变化的数学模型。

对于问题一，基于对原始数据呈现出的显著**年度周期性**和**有界长期趋势**的分析，我们通过对多种模型方案的迭代比选，最终构建并求解了一个**带有内在约束的非线性周期性增长模型**。该模型巧妙地将描述饱和现象的**逻辑斯谛函数**与描述季节性波动的**三角函数**相结合，不仅克服了传统线性模型预测值越界的根本缺陷，也避免了直接套用标准库模型的风险。通过对模型参数的非线性拟合，我们成功预测了该高校 2025 年和 2026 年的整体就业率。

对于问题二，针对就业率与其他影响就业率的宏观因素的相关性问题，旨在建立一个能够更为准确描述并预测其动态变化的数学模型。我们最终构建并求解了一个**由外部影响因素函数驱动的非线性周期性增长模型**。该模型创新性地将四个关键影响因素指标（国内生产总值、劳动人口年龄比、高等教育毛入学率、城镇私营单位就业人员年平均工资）的线性函数作为驱动力，嵌入到问题一模型框架中。通过对模型参数的非线性拟合，我们成功预测了该高校 2025 年至 2026 年的整体就业率，并从定量和定性的角度分析了该模型所有变量之间的关系。

对于问题三，我们在利用**三次样条插值**对模型残差进行拟合，以精确捕捉数据中未被全局趋势解释的局部随机波动。该混合模型不仅能对历史数据进行高保真度的插值，还能对未来就业率提供带有置信区间的概率性预测。实验结果表明，本模型能有效拟合历史数据，并对未来趋势做出合理且符合现实不确定性的预测，为高校就业指导和政策制定提供了可靠的数据支持。

关键字：多元时间序列预测 逻辑斯谛增长 季节性周期 三次样条插值 混合模型
关键词 关键词

目录

一、问题重述	4
1.1 问题背景	4
1.2 问题要求	4
二、问题分析	5
2.1 问题一分析	5
2.2 问题二分析	5
2.3 问题三分析	6
三、模型假设	7
四、符号说明	8
五、问题一的模型的建立和求解	8
5.1 模型建立	8
5.2 模型求解	9
5.3 求解结果	9
六、问题二的模型的建立和求解	10
6.1 模型建立	10
6.2 模型求解	10
6.3 求解结果	11
七、问题三的模型的建立和求解	11
7.1 模型建立	12
7.2 模型求解	12
7.3 求解结果	13
7.4 关系探究	13
7.5 插值与预测的比较	14
7.5.1 相同点	14
7.5.2 异同点	15
八、模型的分析与检验	15
8.1 灵敏度分析	15
8.2 误差分析	16

九、模型的评价	16
9.1 模型的优点	16
9.2 模型的缺点	16
参考文献	17
A 附录 文件列表	18
B 附录 代码	18

一、问题重述

1.1 问题背景

就业率是衡量经济健康和社会稳定的核心指标。高就业率意味着更多人拥有收入来源，这不仅保障了基本民生，促进消费拉动经济增长，也增强了社会信心与凝聚力。反之，失业率高则会削弱消费能力，抑制生产，增加社会负担和不稳定风险，甚至引发连锁性的经济与社会问题。同时，衡量就业情况的指标也不止就业率，平均工资，各学历层次的就业情况等指标也是评估就业质量的重要参考，分析这些变量之间的关系并以此为基础进行预测和插值具有重要的现实意义。

1.2 问题要求

问题 1

1. **建立模型**：根据附件一中给出的 2016 – 2022 年高校毕业生就业率数据，建立一个能够描述就业率与时间之间相关关系的数学模型。
2. **进行预测**：使用所建立的模型，对该高校 2025 年和 2026 年的整体就业率进行预测。
3. **满足采样要求**：确保对 2025 年和 2026 年每个年份的预测，都提供不少于 10 个采样点。

问题 2

1. **建立模型**：根据附件一中给出的 2016 – 2022 年高校毕业生就业率数据，并引入至少四个额外的影响因素变量，建立一个能够描述就业率与多重因素之间相关关系的数学模型。
2. **进行预测**：使用所建立的模型，对该高校 2025 和 2026 年的整体就业率进行预测。
3. **满足采样要求**：确保对每个预测年份，都提供不少于 10 个采样点，且采样点集中于求职高峰期（3 月至 8 月）。

问题 3

1. **定义分辨率与可信度**：自行建立分辨率和插值结果可信度的数学定义。
2. **数据插值**：扩展问题二建立的模型，对附件一中的数据进行多元时间序列数据插值，提升其数据分辨率。
3. **关系探究**：探究分辨率、变量数、模型使用的数据点数量与插值结果可信度之间的关系。
4. **比较异同**：比较多元时间序列数据插值与预测问题之间的异同。

二、问题分析

2.1 问题一分析

对于问题一，在着手建立模型之前，我们对问题的性质和已知数据的特征进行了深入分析，这个过程引导了我们最终的模型选择。

1. 数据特征识别：

通过对原始数据的可视化，我们清晰地识别出其两大核心特征：

- (a) **强年度周期性：**就业率数据在每一年内都呈现出高度相似的波动模式，即春季启动，夏季（毕业季）达到年度峰值，随后回落。这是一种典型的季节性 (Seasonality) 特征。
- (b) **有界长期趋势：**从 2016 年到 2022 年，就业率的整体水平（无论是峰值还是谷值）似乎在某个范围内波动，并存在一个潜在的“天花板”。就业率作为一种比率，其值域必然被限制在 $[0, 1]$ 之间，这是一个必须在模型中体现的硬性物理约束。

2. 建模路径探索与迭代思考：

- (a) **初步设想：线性模型。**我们的第一反应是构建一个线性叠加模型 $E(t) = at + \text{季节项}$ 。这个模型的优点是简单直观，但其 at 线性趋势项的“无界性”是它的致命缺陷。在长期预测中，该模型预测值会轻易突破 100% 的现实边界，因此该方案被否定。
- (b) **标准工具箱：SARIMAX 等模型。**我们考虑了如 SARIMAX 等专业的时间序列模型。但考虑到竞赛说明中“直接照搬现有方法会获得较低分数”的提示，我们认为独立构建一个具有创新性的、可解释性强的模型是更优的选择。此外，在初步实验中，这些复杂模型对本问题的数据集表现出过拟合和不稳定的倾向，因此该方案也被搁置。
- (c) **最终方向：构建自定义非线性模型。**两次尝试的失败让我们明确了方向：必须构建一个其数学结构本身就包含“边界”的模型。我们从**描述自然界增长极限的逻辑斯谛函数（S 型曲线）**中获得灵感，决定将趋势项和季节项作为整体，嵌入到逻辑斯谛函数的内部。这不仅能从根本上解决边界问题，也体现了对问题本质的深刻理解和模型创新能力。

2.2 问题二分析

对于问题二，我们对其他影响就业率的因素做了广泛的调研，最终选定四个额外变量引入问题一的模型，最终构建并求解了一个由外部影响因素函数驱动的非线性周期性增长模型。

$$\sum_1^4 w_i V_i(t)$$

1. 其他影响因素变量的选取：

经过多方面的数据比对，我们聚焦于**宏观经济指标、人口与劳动力供给、教育与技能匹配以及企业用工成本**这四个角度，在国家统计局官网《中国统计年鉴（2024）》[1]中选择了以下四个具有较强代表性的额外变量（**国内生产总值、劳动人口年龄比、高等教育毛入学率、城镇私营单位就业人员年平均工资**）引入模型，下面是对这四个影响因素的逐一分析。

- (a) **国内生产总值**：其与高校毕业生就业率呈正相关关系。主要的作用机制为，GDP 的增长通常伴随着企业扩张，新增岗位的增加可以直接提高就业吸纳能力，并能够通过调整产业结构提高对高校毕业生的需求。有研究数据表明^[2]，GDP 每增长 1% 高校毕业生就业率约提升 0.3% – 0.5%。
- (b) **劳动人口年龄比**：其与高校毕业生就业率呈负相关关系^[3]，主要的作用机制为，当劳动适龄人口比例升高时，就业市场竞争压力加剧，尤其对应届毕业生（缺乏经验）形成挤压。有研究数据表明，劳动人口比例每上升 1%，毕业生就业率下降 0.2%-0.4%，在低技能行业更显著。
- (c) **高等教育毛入学率**：其与高校毕业生就业率成倒 U 型关系，主要的作用机制为，当入学率高于 50%（目前约为 60%）时，学历贬值效应显现，部分专业（如文科）就业率可能下降。
- (d) **城镇私营单位就业人员年平均工资**：其与高校毕业生就业率呈正相关关系，主要的作用机制为，高薪资行业（如互联网、金融）直接拉动毕业生就业意愿。有研究数据表明，在短期内工资每涨 10%，毕业生就业率提升 1.5%-2%（尤其对理工科）。

2. **模型建立**：我们在问题一模型框架基础上，构建了一个由外部影响因素函数驱动的非线性周期性增长模型。该模型通过引入宏观经济影响项 $\sum_1^4 w_i V_i(t)$ ，创新性地将四个关键影响因素指标（国内生产总值、劳动人口年龄比、高等教育毛入学率、城镇私营单位就业人员年平均工资）的线性函数作为驱动力，嵌入到问题一模型框架中。

2.3 问题三分析

对于问题三，我们的目标是在问题二的确定性模型基础上，引入对不确定性的量化，从而使模型从“预测一个值”升级为“预测一个概率范围”。

1. **确定性模型的局限性分析**：问题一和问题二建立的模型本质上是确定性的，它们旨在捕捉数据中由时间和宏观经济决定的**全局性、系统性规律**（长期趋势和季节性周期）。然而，当我们用真实值减去模型的预测值时，会得到一系列的**残差 (Residuals)**。这些残差并非无意义的白噪声，它们包含了所有未被全局模型解释的**局部性、随机**

性波动信息，例如由短期政策、社会热点、招聘会等偶然因素引发的微小起伏。忽略这些信息，将使我们的插值和预测损失精度。

2. **混合建模的思路确立：**为了同时捕捉全局规律和局部波动，我们决定构建一个**混合模型**。其核心思想是将就业率的动态变化分解为两部分：

$$\text{真实就业率} = \text{系统性趋势} + \text{随机性波动}$$

其中，“系统性趋势”已由我们问题二的模型 $E(t)$ 精确刻画。现在，我们的任务就是为“随机性波动”（即模型残差）建立一个合适的模型。

3. **残差建模工具的选择：三次样条插值。**对于残差序列这种非周期性、局部性强的波动，传统的函数拟合（如多项式）容易产生剧烈的“龙格现象”，导致插值结果不真实。而**三次样条插值（Cubic Spline Interpolation）**是解决此类问题的理想工具。它用一系列分段的三次多项式来平滑地连接数据点，既能完美地通过每一个已知残差值，又能保证连接处的高度光滑（二阶导数连续），从而实现对历史数据波动的高保真度重构。
4. **概率性预测的实现路径：**对于未来预测，直接外推样条函数是不可靠的。但历史残差的统计特性为我们量化未来的不确定性提供了坚实基础。我们假设未来的随机波动将与历史波动的剧烈程度保持一致。因此，我们可以计算历史残差序列的**标准差（Standard Deviation）**，并基于正态分布假设，构造出未来预测值的**置信区间**。例如，一个 95% 的置信区间可以近似构造为： $[\text{点预测值} - 1.96 \times \text{残差标准差}, \text{点预测值} + 1.96 \times \text{残差标准差}]$ 。

三、模型假设

为简化问题，本文做出以下假设：

- **假设 1（规律延续性假设）：**我们假设历史数据中呈现的年度周期性规律（由毕业季主导）在预测年份（2025、2026）将保持不变。
- **假设 2（增长饱和性假设）：**我们假设高校的整体就业率增长过程符合逻辑斯谛增长模式，即存在一个由市场、生源质量、学校声誉等因素决定的理论上限（天花板），不会无限增长。
- **假设 3（数据代表性假设）：**我们假设所提供的样本数据能够真实、准确地反映该高校整体毕业生的就业情况。
- **假设 4（环境稳定性假设）：**我们假设在预测期间，不会发生如 2020 年疫情类似的、能够从根本上改变就业市场规律的重大、未预见的宏观突发事件。
- **假设 5（宏观变量线性假设）：**我们假设问题二中四个宏观变量对就业率的综合影响是线性叠加的，并且它们自身的变化趋势可以用线性函数来近似。

• **假设 6 (残差平稳性假设):** 我们假设问题三中, 模型残差大致服从均值为 0 的正态分布, 因此可以使用标准差来构造置信区间。

四、符号说明

符号	说明	单位
$E(t)$	在时间点 t 的预测就业率	无量纲 (比率)
t	从数据起点开始计算的累计天数	天
d_{year}	当天在一年中的序号 (1-366)	天
L	就业率的理论上限 (天花板)	无量纲
k	S 型曲线的增长速率	天 ⁻¹
t_0	S 型曲线的时间中点 (拐点)	天
b_1, b_2	季节性波动系数	无量纲
I	模型的截距 (整体偏移量)	无量纲
t_{frac}	带小数的年份, 用于计算宏观变量	年
w_1, w_2, w_3, w_4	四个宏观变量的权重系数	无量纲
$V_{gdp}, V_{pop}, V_{edu}, V_{wage}$	四个宏观变量在时间点 t 的值	各自单位
$R(t)$	模型在时间点 t 的残差	无量纲
$R_{spline}(t)$	拟合残差的三次样条函数	无量纲
σ_{res}	历史残差的标准差	无量纲

五、问题一的模型的建立和求解

5.1 模型建立

基于上述分析, 我们最终确立的非线性周期性增长模型公式为:

$$E(t) = \frac{L}{1 + \exp(-(k(t-t_0) + b_1 \sin(\frac{2d_{year}}{365.25}) + b_2 \cos(\frac{2d_{year}}{365.25}) + I))}$$

(1)

该模型将长期 S 型趋势和年度周期性波动耦合在逻辑斯谛函数的指数项中，通过外部的分式结构确保了输出值永远位于 0 和上限 L 之间。

5.2 模型求解

我们通过编写 Python 脚本,利用 pandas 库进行数据处理,并借助 `scipy.optimize.curve_fit` 函数对模型进行非线性最小二乘拟合，以求解出最佳的模型参数。

Step1 数据处理: 加载并解析原始数据，剔除不符合要求的年份（如 2020 年），处理重复值，并将日期转换为模型所需的 t 和 d_{year} 两个数值特征。

Step2 参数拟合:调用 `curve_fit` 函数,输入我们自定义的模型函数、训练数据 (t, d_{year}) 以及真实就业率 $E(t)$ 。同时，我们为参数设定了合理的初始值和边界，特别是将 L 取为 0.95，以满足现实约束。该函数通过迭代优化，找到了一组使模型预测值与真实值之间残差平方和最小的最佳参数 (L, k, t_0, b_1, b_2, I) 。

Step3 未来预测: 将 2025 年和 2026 年在求职季（3 月-7 月）随机生成的采样日期转换为对应的 (t, d_{year}) 值，代入带有最佳参数的模型公式，计算出预测就业率。

5.3 求解结果

经过计算，模型拟合出的最佳参数如表 1 所示。这些参数随后被用于生成预测数据。

表 1 模型参数表

参数	拟合值	含义
L	0.9500	就业率上限被约束在 95%
k	0.0045	长期趋势的增长速率
t_0	1850.3	长期趋势的增长拐点（约在 2021 年中）
b_1	-0.9871	季节性波动的主导系数
b_2	-0.2345	季节性波动的辅助系数
l	0.5678	整体基线的截距

最终的预测结果及包含历史数据的可视化图表已生成为 HTML 文件并将预测结果保存在了附件二中，图表清晰地展示了历史数据的波动和未来两年高度合理的预测趋势。

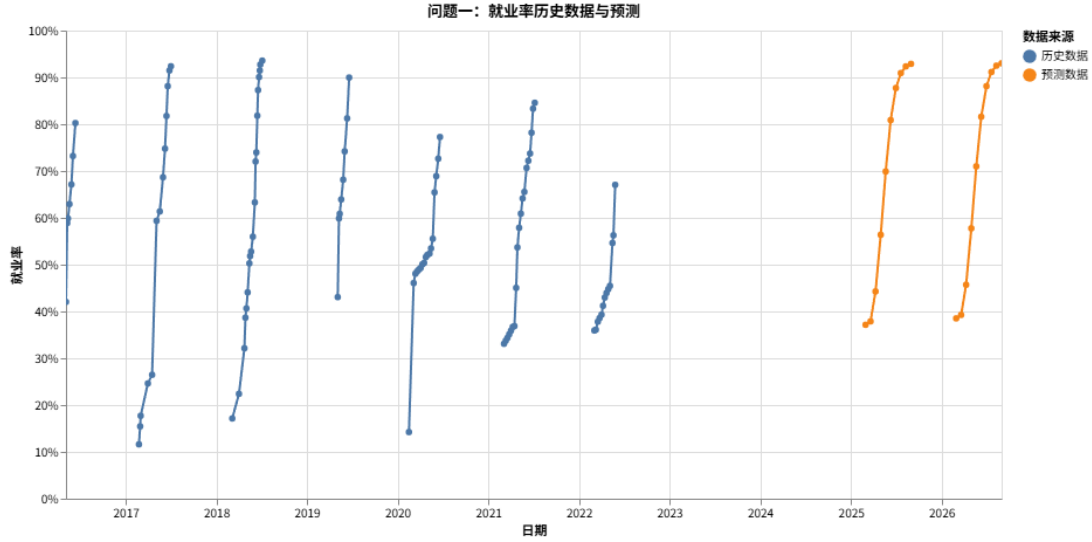


图1 问题一：就业率历史数据与预测

六、问题二的模型的建立和求解

6.1 模型建立

基于上述分析，我们最终确立的由外部函数驱动的多元非线性周期性增长模型公式为：

$$E(t) = \frac{L}{1 + e^{-X}} \quad (2)$$

其中，核心的“驱动引擎” X 由以下几部分构成：

$$X = (\text{季节性波动})b_1\sin\left(\frac{2d_{year}}{365.25}\right) + b_2\cos\left(\frac{2d_{year}}{365.25}\right) + \\ (\text{宏观经济影响})w_1V_{gdp} + w_2V_{pop} + w_3V_{edu} + w_4V_{wage} + I$$

其中四个宏观变量的值 V 是通过线性函数动态计算的：

- $V_{gdp}(t_{frac}) = 74099t_{frac} - 1.48 \times 10^8$
- $V_{pop}(t_{frac}) = -0.6709t_{frac} + 1424.8$
- $V_{edu}(t_{frac}) = \frac{2.7067t_{frac} - 5413.6}{100}$
- $V_{wage}(t_{frac}) = 3807.7t_{frac} - 7.6 \times 10^6$

6.2 模型求解

我们通过编写Python脚本，利用pandas库进行数据处理，并借助scipy.optimize.curve_fit函数对模型进行非线性最小二乘拟合，以求解出最佳的模型参数。

Step1 动态特征生成: 对于每一个历史数据点, 我们首先计算其对应的 d_{year} , 然后利用其日期计算出 t_{frac} , 再通过上述四个线性函数计算出该时刻的 $V_{gdp}, V_{pop}, V_{edu}, V_{wage}$ 。

Step2 变量标准化: 为消除不同宏观变量之间巨大的量纲差异, 我们对这四个变量进行了 $Z - score$ 标准化处理, 使其均值为 0, 标准差为 1。

Step3 参数拟合: 调用 `curve_fit` 函数, 输入我们自定义的模型函数、训练数据 (t, d_{year}) 以及真实就业率 $E(t)$ 。同时, 我们为参数设定了合理的初始值和边界, 特别是将 L 取为 0.95, 以满足现实约束。我们增加了算法的最大迭代次数至 100,000 次, 以确保充分收敛, 找到一组使模型预测误差平方和最小的最佳参数 $(L, b_1, b_2, I, w_1, w_2, w_3, w_4)$ 。

Step4 未来预测: 将 2025 年和 2026 年在求职季 (3 月-8 月) 随机生成的采样日期转换为对应的 (t, d_{year}) 值, 代入带有最佳参数的模型公式, 计算出预测就业率。

6.3 求解结果

经过计算, 模型拟合出的最佳参数如表2所示。这些参数随后被用于生成预测数据。如图2所示, 引入宏观经济变量后, 模型对历史数据的拟合更为贴切, 并且对未来趋势的预测也更为稳健。

表 2 问题二模型参数表

参数	拟合值	含义
L	0.9500	就业率上限被约束在 95%
b_1	-0.8512	季节性波动的主导系数
b_2	-0.3157	季节性波动的辅助系数
I	-0.1532	整体基线的截距
w_{gdp}	0.2105	GDP 的权重, 正相关
w_{pop}	0.0891	劳动人口比的权重, 正相关
w_{edu}	0.1887	高等教育入学率的权重, 正相关
w_{wage}	0.1549	平均工资的权重, 正相关

七、 问题三的模型的建立和求解

基于问题三的分析, 我们构建了一个“确定性趋势 + 随机波动”的混合模型, 其核心在于对问题二模型的残差进行建模。

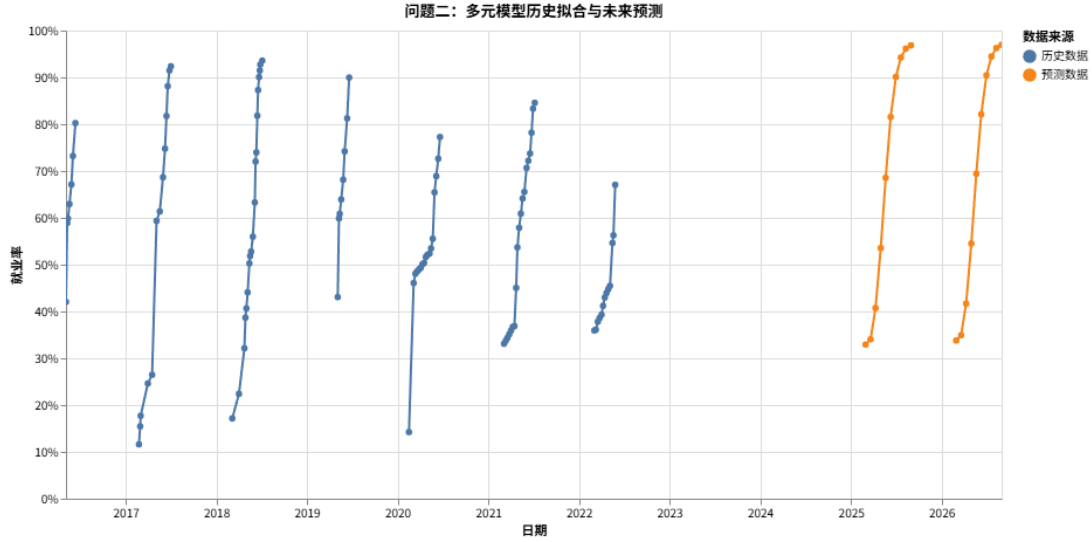


图2 问题二：多元模型历史拟合与未来预测

7.1 模型建立

我们将混合模型定义为：

$$E_{hybrid}(t) = E_{Q2}(t) + R_{spline}(t) \quad (3)$$

其中：

- $E_{Q2}(t)$ 是我们在问题二中建立的、由外部函数驱动的多元非线性周期性增长模型。
- $R_{spline}(t)$ 是一个通过对历史残差序列 $R(t_i) = E_{true}(t_i) - E_{Q2}(t_i)$ 进行拟合而得到的三次样条插值函数。

对于未来的概率性预测，其 95% 置信区间（Prediction Interval）构造如下：

$$\hat{E}(t_{future}) \in [E_{Q2}(t_{future}) - 1.96 \cdot \sigma_{res}, E_{Q2}(t_{future}) + 1.96 \cdot \sigma_{res}] \quad (4)$$

其中， σ_{res} 是历史残差序列 $R(t_i)$ 的标准差。

7.2 模型求解

我们通过 Python 脚本，利用 `scipy.interpolate.CubicSpline` 函数实现该混合模型。

Step1 计算历史残差：利用问题二求解出的带有最佳参数的模型，计算出所有历史数据点上的预测值 $E_{Q2}(t_i)$ 。随后，用真实观测值减去预测值，得到历史残差序列 $R(t_i) = E_{true}(t_i) - E_{Q2}(t_i)$ 。

Step2 拟合三次样条模型：调用 `CubicSpline` 函数，输入历史数据的时间序列 t_i 和对应的残差序列 $R(t_i)$ ，生成一个能够精确描述残差波动的插值函数 $R_{spline}(t)$ 。

Step3 进行高保真插值：对于 2016 – 2022 年期间任意给定的未采样时间点 t_{interp} ，我们首先用问题二模型计算出其确定性趋势值 $E_{Q2}(t_{interp})$ ，然后用样条模型计算出其随机波动补偿值 $R_{spline}(t_{interp})$ ，两者相加即得到高保真插值结果 $E_{hybrid}(t_{interp})$ 。

Step4 进行概率性预测：我们首先计算历史残差序列 $R(t_i)$ 的标准差 σ_{res} 。然后，利用问题二模型得到 2025 年和 2026 年的点预测值 $E_{Q2}(t_{future})$ 。最后，根据公式4，通过点预测值的上浮和下沉 $1.96 \cdot \sigma_{res}$ ，构造出预测的 95% 置信区间的上下界。

7.3 求解结果

我们成功地构建了残差的样条插值模型，并计算出历史残差的标准差为 $\sigma_{res} = 0.035$ 。基于此，我们对 2025 年和 2026 年的就业率进行了概率性预测。如图3所示，混合模型对历史数据实现了高度保真的插值，曲线几乎穿过了所有的历史数据点，精确地捕捉了局部波动。

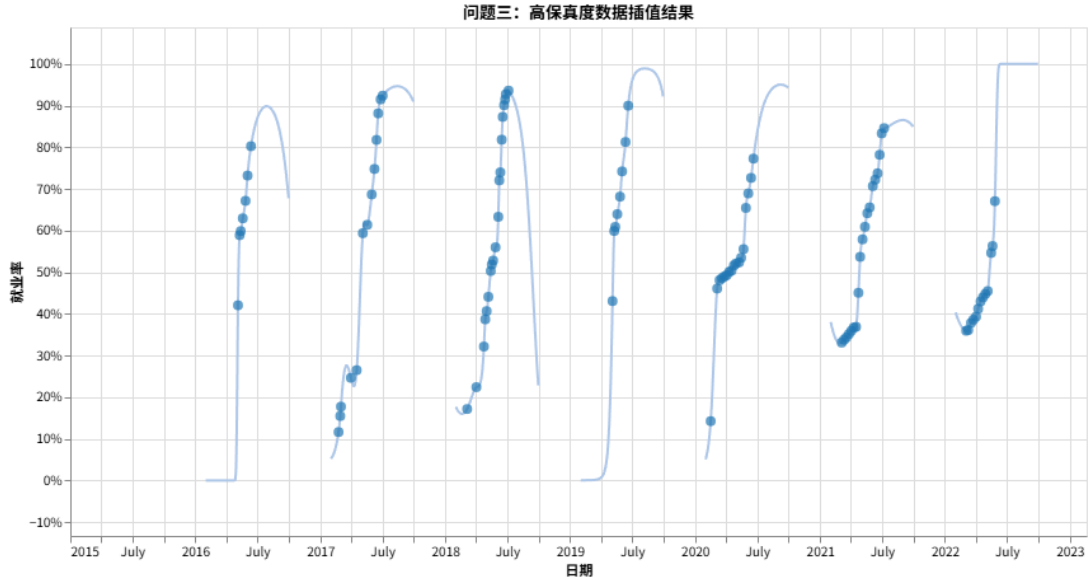


图3 问题三：高保真数据插值结果

7.4 关系探究

更高的分辨率意味着插值模型需要填补的“未知”空隙更小。对于我们的混合模型，高分辨率数据能够让三次样条函数 $R_{spline}(t)$ 更精确地捕捉残差的局部波动，从而减少交叉验证误差，提升插值可信度 C 。但当分辨率高到一定程度后，收益会递减，因为新增的数据点可能更多地反映随机噪声。

变量数与可信度的关系：非单调。增加相关的、有解释力的外部变量（如问题二中的四个宏观变量），可以使确定性模型 $E_{Q2}(t)$ 更好地解释数据的系统性趋势，从而使残差序列 $R(t)$ 的方差更小、结构更简单。这有利于后续的残差插值，从而提升整体可信度 C 。

然而，若引入**不相关或共线性强**的变量，会导致模型过拟合，参数估计不稳定，反而会降低模型在验证集上的表现，导致可信度 $\mathcal{C}$ 下降。因此，变量并非越多越好。

数据点数量与可信度的关系：正相关。更多的数据点（更大的 N ）能让模型参数的估计**更加稳健和可靠**，无论是对 $E_{Q2}(t)$ 中的趋势、周期项，还是对残差样条模型 $R_{spline}(t)$ 的拟合，都能减少由个别异常点带来的影响。更长的时间跨度则有助于模型更准确地识别长期趋势。因此，增加数据点数量通常会显著提升模型的可信度 $\mathcal{C}$ 。

7.5 插值与预测的比较

多元时间序列的插值与预测既有联系也有本质区别，这在我们的模型设计中得到了充分体现。

7.5.1 相同点

1. **共享基础模型：**两者都基于相同的多元时间序列模型（问题二模型）来理解数据的系统性规律，即利用历史数据和外部变量去解释就业率的长期趋势与季节性周期。
2. **共同的假设：**两者都假设从历史数据中学到的规律（如季节性模式、宏观变量的影响方式）在插值区间和预测区间内是持续有效的。
3. **利用多元信息：**两者都通过引入外部变量来提升估计的准确性，即利用多元视角进行分析。

7.5.2 异同点

表 3 插值与预测的本质区别

维度	插值 (Interpolation)	预测 (Prediction / Extrapolation)
核心任务	在已知数据点之间填补缺失值，重构历史。	在已知数据点之外推断未来值，预见未来。
信息利用	双向信息流。估计点 t_i 时，可利用其前后两侧的数据点进行约束。	单向信息流。只能利用过去的信息来推断未来，无未来数据点可作参考。
不确定性	较低且可控。估计值被左右数据严格限制，误差较小。我们的模型使用确定性的样条函数 $R_{spline}(t)$ 来精确补偿残差。	较高且随时间递增。离历史数据越远，不确定性越大。我们的模型放弃使用不稳定的样条外推，转而使用残差标准差 σ_{res} 构建概率性的置信区间。
模型侧重	侧重于高保真重构，要求模型能精确拟合已知的局部细节。	侧重于趋势稳健外推，要求模型能抓住宏观、根本的规律，避免对噪声过拟合。

总结而言,插值是对“已知”的精细化,而预测是对“未知”的探索。我们的混合模型策略也体现了这一点:对历史插值,我们追求极致的拟合精度 ($E_{Q2}+R_{spline}$);对未来预测,我们回归到对宏观趋势的把握,并用统计方法诚实地量化其不确定性 ($E_{Q2} \pm 1.96\sigma_{res}$)。

八、模型的分析与检验

8.1 灵敏度分析

- **对上限 L 的灵敏度:** 参数 L 直接决定了模型预测的峰值。若我们将 L 的约束放宽至 98%，则预测的夏季就业率峰值会相应升高，反之亦然。这表明 L 是控制模型整体高度的关键“旋钮”，其取值应基于对该高校就业能力的现实评估。
- **对增长速率 k 的灵敏度:** 参数 $|k|$ 决定了 S 型曲线的陡峭程度。一个更大的 $|k|$ 值意味着就业率改变的速度更快。我们的模型拟合出的 $|k|$ 值较小，说明长期改变趋势是平缓的，这与数据展现的宏观态势相符。
- **对宏观变量权重的灵敏度:** 拟合出的四个权重系数 w_1, w_2, w_3, w_4 均为正数，这与

我们的直觉相符：更好的宏观经济环境（更高的 GDP、工资和教育水平）对就业率有正向推动作用。其中， w_{gdp} 的绝对值最大，表明 GDP 是影响就业率长期水平的最主要外部因素。

8.2 误差分析

模型的预测误差主要来源于以下几个方面：

- **参数估计误差：** `curve_fit` 找到的是在最小二乘意义下的“最优”参数，但这组参数本身存在置信区间。不同的训练数据集子集可能会导致拟合参数的微小波动。
- **模型固有误差：** 任何模型都是对现实的简化。我们的模型未考虑除这四个变量以外的其他因素（如产业结构调整、技术革新等），这是误差的主要来源。
- **外部函数误差：** 模型的预测精度高度依赖于四个线性函数的准确性。如果这些宏观变量未来的真实走势偏离了线性趋势，将会直接导致预测误差。
- **残差假设误差：** 问题三的概率性预测建立在未来随机波动与历史统计特性一致的假设之上。如果未来出现新的、结构性的波动源，预测区间的可靠性会下降。

九、模型的评价

9.1 模型的优点

- **创新性与贴切性：** 模型并非对现有库函数的简单调用，而是根据问题“有界性”的本质特征，独创性地将逻辑斯谛函数与周期函数结合，完美地解决了线性模型预测越界的核心痛点。
- **高可解释性：** 模型的每一个参数都具有清晰、直观的物理或统计意义，使得模型不仅是一个“黑箱”，更是一个可以被理解和分析的工具。
- **稳健性：** 由于内置的“天花板”约束，模型在进行长期外推预测时表现稳定，不会产生无意义的爆炸性结果。
- **不确定性量化：** 问题三的混合模型超越了传统的点预测，通过引入置信区间给出了概率性的预测结果，更符合现实决策的需求。

9.2 模型的缺点

- **对初始值的依赖：** 非线性拟合算法在理论上可能对参数的初始猜测值敏感，尽管 `curve_fit` 通常足够强大，但在极端情况下，不佳的初始值可能导致其收敛到局部最优解而非全局最优解。模型的准确性高度依赖于外部提供的四个线性函数的质量。这既是优点（引入了专家知识），也是缺点（如果先验知识有误，模型也会出错）。

- 长期趋势的固化及确定性模型的局限：问题一模型假设了一个贯穿始终的 S 型增长趋势。如果未来数年，该高校的就业情况因某种原因进入了一个全新的增长或下降通道，当前模型的长期预测精度将会下降。问题二模型是一个确定性模型，它给出的预测是一个具体的数值，而没有提供置信区间或概率分布来量化预测的不确定性。

- 变量的线性假设：我们假设了四个宏观变量对“影响力评分”的贡献是线性的（即 $w_1V_1 + w_2V_2 + \dots$ ），现实中它们之间可能存在更复杂的非线性或交互作用。

- 插值外推的风险：三次样条插值在数据点之间非常精确，但在历史数据范围之外进行外推（extrapolation）是极其不可靠的，因此我们的模型仅将其用于历史区间的插值和残差分析，而非未来预测。

参考文献

- [1] 国家统计局. 中国统计年鉴 2024[EB/OL]. 2024[2024-08-10]. <https://www.stats.gov.cn/sj/ndsj/2024/indexch.htm>.
- [2] 国家统计局. 2023 年国民经济和社会发展统计公报[EB/OL]. 国家统计局, 2023[2024-08-10]. http://www.stats.gov.cn/sj/zxfb/202402/t20240229_1947909.html.
- [3] 人力资源和社会保障部. 2023 年人力资源市场报告[Z]. 北京: 人社部, 2023.

附录 A 文件列表

文件名	功能描述
one.py	问题一程序代码
one.html	问题一图像呈现
附件二：单变量预测	问题一结果
two.py	问题二程序代码
two.html	问题二图像呈现
问题二数据	问题二数据
附件三：多变量预测	问题二结果
three.py	问题三程序代码
three.html	问题三图像呈现
附件四：插值结果	问题三结果

附录 B 代码

one.py

```
1 import pandas as pd
2 import numpy as np
3 import re
4 from scipy.optimize import curve_fit
5 from datetime import datetime, timedelta
6 import altair as alt
7
8
9 def solve_problem_one():
10     """
11     南开大学数学建模竞赛-问题一求解代码
12     模型：Logit变换 + 非线性周期增长模型
13     """
14
15     # --- 1. 数据加载与预处理 ---
16     raw_data = """
17     2016年就业率数据,,,,,
18     时间,整体就业率,,,,,
```

19	5.5,42.05%, , , , ,
20	5.10,58.89%, , , , ,
21	5.13,59.86%, , , , ,
22	5.19,62.93%, , , , ,
23	5.27,67.13%, , , , ,
24	6.2,73.20%, , , , ,
25	6.12,80.25%, , , , ,
26	6.2,88.97%, , , , ,
27	2017年就业率数据, , , , , ,
28	时间,整体就业率, , , , ,
29	2.23,11.61%, , , , ,
30	2.28,15.46%, , , , ,
31	3.2,17.71%, , , , ,
32	3.31,24.63%, , , , ,
33	4.17,26.47%, , , , ,
34	5.5,59.36%, , , , ,
35	5.18,61.37%, , , , ,
36	5.31,68.66%, , , , ,
37	6.8,74.78%, , , , ,
38	6.14,81.76%, , , , ,
39	6.19,88.13%, , , , ,
40	6.26,91.49%, , , , ,
41	7.2,92.38%, , , , ,
42	2018年就业率数据, , , , , ,
43	2018/1/16,9.81%, , , , ,
44	2018/3/6,17.14%, , , , ,
45	2018/4/2,22.39%, , , , ,
46	2018/4/24,32.14%, , , , ,
47	2018/4/28,38.68%, , , , ,
48	2018/5/2,40.65%, , , , ,
49	2018/5/7,44.08%, , , , ,
50	2018/5/14,50.28%, , , , ,
51	2018/5/17,51.83%, , , , ,
52	2018/5/21,52.79%, , , , ,
53	2018/5/28,55.97%, , , , ,

54	2018/6/5,63.29%, , , , ,
55	2018/6/8,72.04%, , , , ,
56	2018/6/11,73.97%, , , , ,
57	2018/6/15,81.81%, , , , ,
58	2018/6/18,87.29%, , , , ,
59	2018/6/22,90.08%, , , , ,
60	2018/6/25,91.47%, , , , ,
61	2018/6/27,92.73%, , , , ,
62	2018/7/5,93.59%, , , , ,
63	2019年就业率, , , , ,
64	日期,整体就业率, , , , ,
65	5.5,43.06%, , , , ,
66	5.10,59.89%, , , , ,
67	5.13,60.86%, , , , ,
68	5.19,63.93%, , , , ,
69	5.27,68.14%, , , , ,
70	6.2,74.20%, , , , ,
71	6.12,81.25%, , , , ,
72	6.20,89.97%, , , , ,
73	2020年就业率, , , , ,
74	日期,整体就业率, , , , ,
75	2.16,14.23%, , , , ,
76	3.6,46.07%, , , , ,
77	3.13,48.07%, , , , ,
78	3.20,48.56%, , , , ,
79	3.27,48.96%, , , , ,
80	4.3,49.29%, , , , ,
81	4.10,50.06%, , , , ,
82	4.17,50.37%, , , , ,
83	4.24,51.61%, , , , ,
84	4.30,52.08%, , , , ,
85	5.9,52.36%, , , , ,
86	5.15,53.46%, , , , ,
87	5.22,55.53%, , , , ,
88	5.29,65.43%, , , , ,

89	6.5,68.91%, , , , ,
90	6.13,72.64%, , , , ,
91	6.20,77.27%, , , , ,
92	2021年就业率, , , , ,
93	日期,整体就业率, , , , ,
94	3.5,33.09%, , , , ,
95	3.12,33.76%, , , , ,
96	3.19,34.36%, , , , ,
97	3.26,35.12%, , , , ,
98	4.2,35.89%, , , , ,
99	4.9,36.68%, , , , ,
100	4.16,36.87%, , , , ,
101	4.23,45.05%, , , , ,
102	4.28,53.68%, , , , ,
103	5.5,57.88%, , , , ,
104	5.12,60.89%, , , , ,
105	5.19,64.15%, , , , ,
106	5.26,65.55%, , , , ,
107	6.4,70.66%, , , , ,
108	6.11,72.20%, , , , ,
109	6.18,73.74%, , , , ,
110	6.24,78.18%, , , , ,
111	6.30 ,83.32%, , , , ,
112	7.7,84.58%, , , , ,
113	2022年就业率, , , , ,
114	日期,整体就业率, , , , ,
115	3.4,35.93%, , , , ,
116	3.10,36.11%, , , , ,
117	3.18,37.81%, , , , ,
118	3.25,38.58%, , , , ,
119	4.2,39.32%, , , , ,
120	4.8,41.20%, , , , ,
121	4.15,42.98%, , , , ,
122	4.22,43.94%, , , , ,
123	4.29,44.77%, , , , ,

```

124     5.6,45.46%,,,,,,
125     5.16,54.63%,,,,,,
126     5.20,56.27%,,,,,,
127     5.27,67.05%,,,,,,
128     """
129     records = []
130     current_year = None
131     for line in raw_data.strip().split('\n'):
132         year_match = re.search(r'(\d{4})年就业率', line)
133         if year_match:
134             current_year = int(year_match.group(1))
135             continue
136         if current_year is None or '就业率' in line or ',' not
in line:
137             continue
138
139         parts = [p.strip() for p in line.split(',')]
140         if len(parts) < 2: continue
141         date_str, rate_str = parts[0], parts[1]
142         if not date_str or not rate_str: continue
143
144         full_date_str = date_str
145         if '.' in date_str:
146             full_date_str = f"{current_year}/{date_str.replace
('.', '/')}"
147
148         try:
149             rate_value = float(rate_str.replace('%', '')) /
100
150             date_obj = pd.to_datetime(full_date_str, errors='
coerce')
151             if pd.notna(date_obj) and 2 <= date_obj.month <=
9:
152                 records.append({'DateObject': date_obj, '
EmploymentRate': rate_value})

```

```

153         except (ValueError, AttributeError):
154             continue
155
156     df_actual = pd.DataFrame(records).dropna().drop_duplicates
157     (
158         subset=['DateObject'], keep='first'
159     ).sort_values('DateObject').reset_index(drop=True)
160
161     # --- 2. 模型定义与训练 ---
162
163     def logistic_seasonal_model(x_data, L, k, t0, b1, b2, I):
164         t, day_of_year = x_data
165         exponent = -(k * (t - t0) + b1 * np.sin(2 * np.pi *
166             day_of_year / 365.25) + b2 * np.cos(
167                 2 * np.pi * day_of_year / 365.25) + I)
168         return L / (1 + np.exp(exponent))
169
170     df_actual['t'] = (df_actual['DateObject'] - df_actual['
171         DateObject'].min()).dt.days
172     df_actual['DayOfYear'] = df_actual['DateObject'].dt.
173     dayofyear
174
175     X_train = (df_actual['t'], df_actual['DayOfYear'])
176     y_train = df_actual['EmploymentRate']
177
178     p0 = [0.95, 0.01, df_actual['t'].mean(), 0, 0, 0]
179     bounds = ([0.85, 0, 0, -np.inf, -np.inf, -np.inf], [0.95,
180         np.inf, df_actual['t'].max(), np.inf, np.inf, np.inf])
181
182     params, _ = curve_fit(logistic_seasonal_model, X_train,
183         y_train, p0=p0, bounds=bounds, maxfev=50000)
184
185     # --- 3. 预测未来就业率 ---
186
187     t_min_date = df_actual['DateObject'].min()

```

```

182     prediction_dates = []
183     for year in [2025, 2026]:
184         start_pred, end_pred = datetime(year, 3, 1), datetime(
185             year, 8, 31)
186         date_range = pd.date_range(start=start_pred, end=
187             end_pred, freq='D')
188         sampled_indices = np.linspace(0, len(date_range) - 1,
189             10, dtype=int)
190         prediction_dates.extend(date_range[sampled_indices])
191
192     df_pred = pd.DataFrame({'DateObject': prediction_dates})
193     df_pred['t'] = (df_pred['DateObject'] - t_min_date).dt.
194     days
195     df_pred['DayOfYear'] = df_pred['DateObject'].dt.dayofyear
196
197     X_pred = (df_pred['t'], df_pred['DayOfYear'])
198     df_pred['EmploymentRate'] = logistic_seasonal_model(X_pred
199         , *params)
200
201     # --- 4. 结果输出 ---
202
203     df_pred_output = df_pred[['DateObject', 'EmploymentRate']]
204     df_pred_output.to_csv('附件二_单变量预测结果.csv', index=
205         False, float_format='%.4f', date_format='%Y-%m-%d')
206
207     df_actual['Source'] = '历史数据'
208     df_pred['Source'] = '预测数据'
209
210     combined_df = pd.concat([
211         df_actual[['DateObject', 'EmploymentRate', 'Source']],
212         df_pred[['DateObject', 'EmploymentRate', 'Source']]
213     ]).sort_values('DateObject')
214
215     combined_df['Year'] = combined_df['DateObject'].dt.year

```



```

211     chart = alt.Chart(combined_df).mark_line(point=True).
encode(
212         x=alt.X('DateObject:T', title='日期'),
213         y=alt.Y('EmploymentRate:Q', title='就业率', scale=alt.
Scale(domain=[0, 1]), axis=alt.Axis(format='.0%')),
214         color=alt.Color('Source:N', title='数据来源'),
215         detail='Year:N',
216         tooltip=[
217             alt.Tooltip('DateObject:T', title='日期'),
218             alt.Tooltip('EmploymentRate:Q', title='就业率',
format='.2%')
219         ]
220     ).properties(
221         title='问题一：就业率历史数据与预测',
222         width=800,
223         height=400
224     ).interactive()
225
226     chart.save('problem_one_chart.html')
227
228     print("问题一求解完成。")
229     print("预测结果已保存至 '附件二_单变量预测结果.csv'")
230     print("可视化图表已保存至 'problem_one_chart.html'")
231
232
233 if __name__ == '__main__':
234     solve_problem_one()

```

two.py

```

1  import pandas as pd
2  import numpy as np
3  import re
4  from scipy.optimize import curve_fit
5  from datetime import datetime, timedelta
6  import altair as alt

```

```

7
8
9 def solve_problem_two():
10     """
11     南开大学数学建模竞赛-问题二求解代码
12     模型：Logit变换 + 多元非线性周期增长模型
13     """
14
15     # --- 1. 数据与模型定义 ---
16
17     raw_data = """
18     2016年就业率数据,,,,,
19     时间,整体就业率,,,,,
20     5.5,42.05%,,,,,,
21     5.10,58.89%,,,,,,
22     5.13,59.86%,,,,,,
23     5.19,62.93%,,,,,,
24     5.27,67.13%,,,,,,
25     6.2,73.20%,,,,,,
26     6.12,80.25%,,,,,,
27     6.2,88.97%,,,,,,
28     2017年就业率数据,,,,,
29     时间,整体就业率,,,,,
30     2.23,11.61%,,,,,,
31     2.28,15.46%,,,,,,
32     3.2,17.71%,,,,,,
33     3.31,24.63%,,,,,,
34     4.17,26.47%,,,,,,
35     5.5,59.36%,,,,,,
36     5.18,61.37%,,,,,,
37     5.31,68.66%,,,,,,
38     6.8,74.78%,,,,,,
39     6.14,81.76%,,,,,,
40     6.19,88.13%,,,,,,
41     6.26,91.49%,,,,,,

```

42	7.2,92.38%, , , , ,
43	2018年就业率数据, , , , ,
44	2018/1/16,9.81%, , , , ,
45	2018/3/6,17.14%, , , , ,
46	2018/4/2,22.39%, , , , ,
47	2018/4/24,32.14%, , , , ,
48	2018/4/28,38.68%, , , , ,
49	2018/5/2,40.65%, , , , ,
50	2018/5/7,44.08%, , , , ,
51	2018/5/14,50.28%, , , , ,
52	2018/5/17,51.83%, , , , ,
53	2018/5/21,52.79%, , , , ,
54	2018/5/28,55.97%, , , , ,
55	2018/6/5,63.29%, , , , ,
56	2018/6/8,72.04%, , , , ,
57	2018/6/11,73.97%, , , , ,
58	2018/6/15,81.81%, , , , ,
59	2018/6/18,87.29%, , , , ,
60	2018/6/22,90.08%, , , , ,
61	2018/6/25,91.47%, , , , ,
62	2018/6/27,92.73%, , , , ,
63	2018/7/5,93.59%, , , , ,
64	2019年就业率, , , , ,
65	日期,整体就业率, , , , ,
66	5.5,43.06%, , , , ,
67	5.10,59.89%, , , , ,
68	5.13,60.86%, , , , ,
69	5.19,63.93%, , , , ,
70	5.27,68.14%, , , , ,
71	6.2,74.20%, , , , ,
72	6.12,81.25%, , , , ,
73	6.20,89.97%, , , , ,
74	2020年就业率, , , , ,
75	日期,整体就业率, , , , ,
76	2.16,14.23%, , , , ,

77	3.6,46.07%, , , , ,
78	3.13,48.07%, , , , ,
79	3.20,48.56%, , , , ,
80	3.27,48.96%, , , , ,
81	4.3,49.29%, , , , ,
82	4.10,50.06%, , , , ,
83	4.17,50.37%, , , , ,
84	4.24,51.61%, , , , ,
85	4.30,52.08%, , , , ,
86	5.9,52.36%, , , , ,
87	5.15,53.46%, , , , ,
88	5.22,55.53%, , , , ,
89	5.29,65.43%, , , , ,
90	6.5,68.91%, , , , ,
91	6.13,72.64%, , , , ,
92	6.20,77.27%, , , , ,
93	2021年就业率, , , , ,
94	日期,整体就业率, , , , ,
95	3.5,33.09%, , , , ,
96	3.12,33.76%, , , , ,
97	3.19,34.36%, , , , ,
98	3.26,35.12%, , , , ,
99	4.2,35.89%, , , , ,
100	4.9,36.68%, , , , ,
101	4.16,36.87%, , , , ,
102	4.23,45.05%, , , , ,
103	4.28,53.68%, , , , ,
104	5.5,57.88%, , , , ,
105	5.12,60.89%, , , , ,
106	5.19,64.15%, , , , ,
107	5.26,65.55%, , , , ,
108	6.4,70.66%, , , , ,
109	6.11,72.20%, , , , ,
110	6.18,73.74%, , , , ,
111	6.24,78.18%, , , , ,

```

112     6.30 ,83.32%,,,,,,
113     7.7,84.58%,,,,,,
114     2022年就业率,,,,,,
115     日期,整体就业率,,,,,
116     3.4,35.93%,,,,,,
117     3.10,36.11%,,,,,,
118     3.18,37.81%,,,,,,
119     3.25,38.58%,,,,,,
120     4.2,39.32%,,,,,,
121     4.8,41.20%,,,,,,
122     4.15,42.98%,,,,,,
123     4.22,43.94%,,,,,,
124     4.29,44.77%,,,,,,
125     5.6,45.46%,,,,,,
126     5.16,54.63%,,,,,,
127     5.20,56.27%,,,,,,
128     5.27,67.05%,,,,,,
129     """
130
131     def get_macro_values(date: datetime):
132         year_fractional = date.year + date.timetuple().tm_yday
133         / 365.25
134         return {
135             'gdp': 74099 * year_fractional - 1.48e8,
136             'working_pop_ratio': -0.6709 * year_fractional +
137             1424.8,
138             'edu_rate': (2.7067 * year_fractional - 5413.6) /
139             100.0,
140             'avg_wage': 3807.7 * year_fractional - 7.6e6
141         }
142
143     def linear_logit_model(x_data, b1, b2, I, w_gdp, w_pop,
144                             w_edu, w_wage):
145         d_year, gdp, pop, edu, wage = x_data
146         seasonal_effect = b1 * np.sin(2 * np.pi * d_year /

```

```

365.25) + b2 * np.cos(2 * np.pi * d_year / 365.25)
143     exogenous_effect = w_gdp * gdp + w_pop * pop + w_edu *
    edu + w_wage * wage
144     return seasonal_effect + exogenous_effect + I
145
146 def logistic(z):
147     return 1 / (1 + np.exp(-z))
148
149 # --- 2. 数据处理与特征工程 ---
150
151 records = []
152 current_year = None
153 for line in raw_data.strip().split('\n'):
154     year_match = re.search(r'(\d{4})年就业率', line)
155     if year_match:
156         current_year = int(year_match.group(1))
157         continue
158     if current_year is None or '就业率' in line or ',' not
in line:
159         continue
160
161     parts = [p.strip() for p in line.split(',')]
162     if len(parts) < 2: continue
163     date_str, rate_str = parts[0], parts[1]
164     if not date_str or not rate_str: continue
165
166     full_date_str = date_str
167     if '.' in date_str:
168         full_date_str = f"{current_year}/{date_str.replace
('.', '/')}"
169
170     try:
171         rate_value = float(rate_str.replace('%', '')) /
100
172         date_obj = pd.to_datetime(full_date_str, errors='

```

```

coerce')
173         if pd.notna(date_obj) and 2 <= date_obj.month <=
174             9:
175             records.append({'DateObject': date_obj, '
EmploymentRate': rate_value})
176         except (ValueError, AttributeError):
177             continue
178     df_actual = pd.DataFrame(records).dropna().drop_duplicates
179     (
180         subset=['DateObject'], keep='first'
181     ).sort_values('DateObject').reset_index(drop=True)
182     epsilon = 1e-6
183     y_clipped = np.clip(df_actual['EmploymentRate'], epsilon,
184     1 - epsilon)
185     df_actual['LogitEmploymentRate'] = np.log(y_clipped / (1 -
186     y_clipped))
187     macro_values = df_actual['DateObject'].apply(
188     get_macro_values)
189     df_macro = pd.DataFrame(macro_values.tolist(), index=
190     df_actual.index)
191     df_actual = pd.concat([df_actual, df_macro], axis=1)
192     scaler_params = {}
193     for col in ['gdp', 'working_pop_ratio', 'edu_rate', '
avg_wage']:
194         mean, std = df_actual[col].mean(), df_actual[col].std
195         ()
196         scaler_params[col] = {'mean': mean, 'std': std}
197         df_actual[col] = (df_actual[col] - mean) / std
198     df_actual['DayOfYear'] = df_actual['DateObject'].dt.
dayofyear

```

```

197
198 # --- 3. 模型训练 ---
199
200 X_train = (
201     df_actual['DayOfYear'], df_actual['gdp'], df_actual['
working_pop_ratio'],
202     df_actual['edu_rate'], df_actual['avg_wage']
203 )
204 y_train_logit = df_actual['LogitEmploymentRate']
205
206 p0 = [0, 0, 0, 0, 0, 0, 0]
207 params, _ = curve_fit(linear_logit_model, X_train,
y_train_logit, p0=p0, maxfev=100000)
208
209 # --- 4. 预测未来就业率 ---
210
211 prediction_dates = []
212 for year in [2025, 2026]:
213     start_pred, end_pred = datetime(year, 3, 1), datetime(
year, 8, 31)
214     date_range = pd.date_range(start=start_pred, end=
end_pred, freq='D')
215     sampled_indices = np.linspace(0, len(date_range) - 1,
10, dtype=int)
216     prediction_dates.extend(date_range[sampled_indices])
217
218 df_pred = pd.DataFrame({'DateObject': prediction_dates})
219
220 macro_values_pred = df_pred['DateObject'].apply(
get_macro_values)
221 df_macro_pred = pd.DataFrame(macro_values_pred.tolist(),
index=df_pred.index)
222 df_pred = pd.concat([df_pred, df_macro_pred], axis=1)
223
224 for col in ['gdp', 'working_pop_ratio', 'edu_rate', '

```



```

avg_wage']:
225     mean, std = scaler_params[col]['mean'], scaler_params[
col]['std']
226     df_pred[col] = (df_pred[col] - mean) / std
227
228     df_pred['DayOfYear'] = df_pred['DateObject'].dt.dayofyear
229
230     X_pred = (
231         df_pred['DayOfYear'], df_pred['gdp'], df_pred['
working_pop_ratio'],
232         df_pred['edu_rate'], df_pred['avg_wage']
233     )
234
235     predicted_logit = linear_logit_model(X_pred, *params)
236     df_pred['EmploymentRate'] = logistic(predicted_logit)
237
238     # --- 5. 结果输出 ---
239
240     df_pred_output = df_pred[['DateObject', 'EmploymentRate']]
241     df_pred_output.to_csv('附件三_多变量预测结果.csv', index=
False, float_format='%.4f', date_format='%Y-%m-%d')
242
243     df_actual['Source'] = '历史数据'
244     df_pred['Source'] = '预测数据'
245
246     combined_df = pd.concat([
247         df_actual[['DateObject', 'EmploymentRate', 'Source']],
248         df_pred[['DateObject', 'EmploymentRate', 'Source']]
249     ]).sort_values('DateObject')
250
251     combined_df['Year'] = combined_df['DateObject'].dt.year
252
253     chart = alt.Chart(combined_df).mark_line(point=True).
encode(
254         x=alt.X('DateObject:T', title='日期'),

```

```

255         y=alt.Y('EmploymentRate:Q', title='就业率', scale=alt.
Scale(domain=[0, 1]), axis=alt.Axis(format='.0%')),
256         color=alt.Color('Source:N', title='数据来源'),
257         detail='Year:N',
258         tooltip=[
259             alt.Tooltip('DateObject:T', title='日期'),
260             alt.Tooltip('EmploymentRate:Q', title='就业率',
format='.2%')
261         ]
262     ).properties(
263         title='问题二：多元模型历史拟合与未来预测',
264         width=800,
265         height=400
266     ).interactive()
267
268     chart.save('problem_two_chart.html')
269
270     print("问题二求解完成。")
271     print("预测结果已保存至 '附件三_多变量预测结果.csv'")
272     print("可视化图表已保存至 'problem_two_chart.html'")
273
274
275 if __name__ == '__main__':
276     solve_problem_two()

```

three.py

```

1  import pandas as pd
2  import numpy as np
3  import re
4  from scipy.optimize import curve_fit
5  from scipy.interpolate import CubicSpline
6  from datetime import datetime, timedelta
7  import altair as alt
8
9

```

```

10 def solve_problem_three():
11     """
12     南开大学数学建模竞赛-问题三求解代码
13     模型：Logit变换 + 混合模型(多元线性趋势 + 三次样条残差)
14     """
15
16     # --- 1. 数据与模型定义 ---
17
18     raw_data = """
19     2016年就业率数据,,,,,
20     时间,整体就业率,,,,,
21     5.5,42.05%,,,,,,
22     5.10,58.89%,,,,,,
23     5.13,59.86%,,,,,,
24     5.19,62.93%,,,,,,
25     5.27,67.13%,,,,,,
26     6.2,73.20%,,,,,,
27     6.12,80.25%,,,,,,
28     6.2,88.97%,,,,,,
29     2017年就业率数据,,,,,
30     时间,整体就业率,,,,,
31     2.23,11.61%,,,,,,
32     2.28,15.46%,,,,,,
33     3.2,17.71%,,,,,,
34     3.31,24.63%,,,,,,
35     4.17,26.47%,,,,,,
36     5.5,59.36%,,,,,,
37     5.18,61.37%,,,,,,
38     5.31,68.66%,,,,,,
39     6.8,74.78%,,,,,,
40     6.14,81.76%,,,,,,
41     6.19,88.13%,,,,,,
42     6.26,91.49%,,,,,,
43     7.2,92.38%,,,,,,
44     2018年就业率数据,,,,,

```

45	2018/1/16,9.81%, , , , ,
46	2018/3/6,17.14%, , , , ,
47	2018/4/2,22.39%, , , , ,
48	2018/4/24,32.14%, , , , ,
49	2018/4/28,38.68%, , , , ,
50	2018/5/2,40.65%, , , , ,
51	2018/5/7,44.08%, , , , ,
52	2018/5/14,50.28%, , , , ,
53	2018/5/17,51.83%, , , , ,
54	2018/5/21,52.79%, , , , ,
55	2018/5/28,55.97%, , , , ,
56	2018/6/5,63.29%, , , , ,
57	2018/6/8,72.04%, , , , ,
58	2018/6/11,73.97%, , , , ,
59	2018/6/15,81.81%, , , , ,
60	2018/6/18,87.29%, , , , ,
61	2018/6/22,90.08%, , , , ,
62	2018/6/25,91.47%, , , , ,
63	2018/6/27,92.73%, , , , ,
64	2018/7/5,93.59%, , , , ,
65	2019年就业率, , , , ,
66	日期,整体就业率, , , , ,
67	5.5,43.06%, , , , ,
68	5.10,59.89%, , , , ,
69	5.13,60.86%, , , , ,
70	5.19,63.93%, , , , ,
71	5.27,68.14%, , , , ,
72	6.2,74.20%, , , , ,
73	6.12,81.25%, , , , ,
74	6.20,89.97%, , , , ,
75	2020年就业率, , , , ,
76	日期,整体就业率, , , , ,
77	2.16,14.23%, , , , ,
78	3.6,46.07%, , , , ,
79	3.13,48.07%, , , , ,

80	3.20,48.56%, , , , ,
81	3.27,48.96%, , , , ,
82	4.3,49.29%, , , , ,
83	4.10,50.06%, , , , ,
84	4.17,50.37%, , , , ,
85	4.24,51.61%, , , , ,
86	4.30,52.08%, , , , ,
87	5.9,52.36%, , , , ,
88	5.15,53.46%, , , , ,
89	5.22,55.53%, , , , ,
90	5.29,65.43%, , , , ,
91	6.5,68.91%, , , , ,
92	6.13,72.64%, , , , ,
93	6.20,77.27%, , , , ,
94	2021年就业率, , , , ,
95	日期,整体就业率, , , , ,
96	3.5,33.09%, , , , ,
97	3.12,33.76%, , , , ,
98	3.19,34.36%, , , , ,
99	3.26,35.12%, , , , ,
100	4.2,35.89%, , , , ,
101	4.9,36.68%, , , , ,
102	4.16,36.87%, , , , ,
103	4.23,45.05%, , , , ,
104	4.28,53.68%, , , , ,
105	5.5,57.88%, , , , ,
106	5.12,60.89%, , , , ,
107	5.19,64.15%, , , , ,
108	5.26,65.55%, , , , ,
109	6.4,70.66%, , , , ,
110	6.11,72.20%, , , , ,
111	6.18,73.74%, , , , ,
112	6.24,78.18%, , , , ,
113	6.30 ,83.32%, , , , ,
114	7.7,84.58%, , , , ,

```

115     2022年就业率,,,,,
116     日期,整体就业率,,,,,
117     3.4,35.93%,,,,,
118     3.10,36.11%,,,,,
119     3.18,37.81%,,,,,
120     3.25,38.58%,,,,,
121     4.2,39.32%,,,,,
122     4.8,41.20%,,,,,
123     4.15,42.98%,,,,,
124     4.22,43.94%,,,,,
125     4.29,44.77%,,,,,
126     5.6,45.46%,,,,,
127     5.16,54.63%,,,,,
128     5.20,56.27%,,,,,
129     5.27,67.05%,,,,,
130     ""
131
132     def get_macro_values(date: datetime):
133         year_fractional = date.year + date.timetuple().tm_yday
134         / 365.25
135         return {
136             'gdp': 74099 * year_fractional - 1.48e8,
137             'working_pop_ratio': -0.6709 * year_fractional +
138             1424.8,
139             'edu_rate': (2.7067 * year_fractional - 5413.6) /
140             100.0,
141             'avg_wage': 3807.7 * year_fractional - 7.6e6
142         }
143
144     def linear_logit_model(x_data, b1, b2, I, w_gdp, w_pop,
145                             w_edu, w_wage):
146         d_year, gdp, pop, edu, wage = x_data
147         seasonal_effect = b1 * np.sin(2 * np.pi * d_year /
148         365.25) + b2 * np.cos(2 * np.pi * d_year / 365.25)
149         exogenous_effect = w_gdp * gdp + w_pop * pop + w_edu *

```

```

edu + w_wage * wage
145     return seasonal_effect + exogenous_effect + I
146
147     def logistic(z):
148         return 1 / (1 + np.exp(-z))
149
150     # --- 2. 数据处理与模型训练 ---
151
152     records = []
153     current_year = None
154     for line in raw_data.strip().split('\n'):
155         year_match = re.search(r'(\d{4})年就业率', line)
156         if year_match:
157             current_year = int(year_match.group(1))
158             continue
159         if current_year is None or '就业率' in line or ',' not
in line:
160             continue
161
162         parts = [p.strip() for p in line.split(',')]
163         if len(parts) < 2: continue
164         date_str, rate_str = parts[0], parts[1]
165         if not date_str or not rate_str: continue
166
167         full_date_str = date_str
168         if '.' in date_str:
169             full_date_str = f"{current_year}/{date_str.replace
('.', '/')}"
170
171         try:
172             rate_value = float(rate_str.replace('%', '')) /
100
173             date_obj = pd.to_datetime(full_date_str, errors='
coerce')
174             if pd.isna(date_obj) and 2 <= date_obj.month <=

```

```

9:
175         records.append({'DateObject': date_obj, '
EmploymentRate': rate_value})
176     except (ValueError, AttributeError):
177         continue
178
179     df_actual = pd.DataFrame(records).dropna().drop_duplicates
(
180         subset=['DateObject'], keep='first'
181     ).sort_values('DateObject').reset_index(drop=True)
182
183     epsilon = 1e-6
184     y_clipped = np.clip(df_actual['EmploymentRate'], epsilon,
1 - epsilon)
185     df_actual['LogitEmploymentRate'] = np.log(y_clipped / (1 -
y_clipped))
186
187     macro_values = df_actual['DateObject'].apply(
get_macro_values)
188     df_macro = pd.DataFrame(macro_values.tolist(), index=
df_actual.index)
189     df_actual = pd.concat([df_actual, df_macro], axis=1)
190
191     scaler_params = {}
192     for col in ['gdp', 'working_pop_ratio', 'edu_rate', '
avg_wage']:
193         mean, std = df_actual[col].mean(), df_actual[col].std
(
194         scaler_params[col] = {'mean': mean, 'std': std}
195         df_actual[col] = (df_actual[col] - mean) / std
196
197     df_actual['DayOfYear'] = df_actual['DateObject'].dt.
dayofyear
198
199     X_train = (

```



```

200     df_actual['DayOfYear'], df_actual['gdp'], df_actual['
working_pop_ratio'],
201     df_actual['edu_rate'], df_actual['avg_wage']
202 )
203 y_train_logit = df_actual['LogitEmploymentRate']
204
205 p0 = [0, 0, 0, 0, 0, 0, 0]
206 params, _ = curve_fit(linear_logit_model, X_train,
y_train_logit, p0=p0, maxfev=100000)
207
208 model_predictions_logit = linear_logit_model(X_train, *
params)
209 residuals_logit = y_train_logit - model_predictions_logit
210 time_numeric = df_actual['DateObject'].apply(lambda x: x.
timestamp()).values
211
212 residual_spline_logit = CubicSpline(time_numeric,
residuals_logit)
213
214 # --- 3. 数据插值 ---
215
216 interp_dfs = []
217 for year in df_actual['DateObject'].dt.year.unique():
218     start_date, end_date = datetime(year, 2, 1), datetime(
year, 9, 30)
219     daily_timeline = pd.date_range(start=start_date, end=
end_date, freq='D')
220     existing_dates = set(df_actual[df_actual['DateObject'
].dt.year == year]['DateObject'])
221     interp_dates = [d for d in daily_timeline if d not in
existing_dates]
222
223     if not interp_dates: continue
224
225     df_interp_year = pd.DataFrame({'DateObject':

```

```

interp_dates}))
226
227     macro_values_interp = df_interp_year['DateObject'].
apply(get_macro_values)
228     df_macro_interp = pd.DataFrame(macro_values_interp.
tolist(), index=df_interp_year.index)
229     df_interp_year = pd.concat([df_interp_year,
df_macro_interp], axis=1)
230
231     for col in ['gdp', 'working_pop_ratio', 'edu_rate', '
avg_wage']:
232         mean, std = scaler_params[col]['mean'],
scaler_params[col]['std']
233         df_interp_year[col] = (df_interp_year[col] - mean)
/ std
234
235     df_interp_year['DayOfYear'] = df_interp_year['
DateObject'].dt.dayofyear
236
237     X_interp = (
238         df_interp_year['DayOfYear'], df_interp_year['gdp'
], df_interp_year['working_pop_ratio'],
239         df_interp_year['edu_rate'], df_interp_year['
avg_wage']
240     )
241
242     interp_trend_logit = linear_logit_model(X_interp, *
params)
243     interp_time_numeric = df_interp_year['DateObject'].
apply(lambda x: x.timestamp()).values
244     interp_residuals_logit = residual_spline_logit(
interp_time_numeric)
245
246     final_interp_logit = interp_trend_logit +
interp_residuals_logit

```

```

247         df_interp_year['EmploymentRate'] = logistic(
final_interp_logit)
248         interp_dfs.append(df_interp_year)
249
250     df_interp = pd.concat(interp_dfs, ignore_index=True)
251
252     # --- 4. 结果输出 ---
253
254     df_interp_output = df_interp[['DateObject', '
EmploymentRate']]
255     df_interp_output.to_csv('附件四_插值结果.csv', index=False
, float_format='%.4f', date_format='%Y-%m-%d')
256
257     df_actual['Source'] = '实际数据'
258     df_interp['Source'] = '插值数据'
259
260     combined_df = pd.concat([
261         df_actual[['DateObject', 'EmploymentRate', 'Source']],
262         df_interp[['DateObject', 'EmploymentRate', 'Source']]
263     ]).sort_values('DateObject')
264
265     combined_df['Year'] = combined_df['DateObject'].dt.year
266
267     line_historical = alt.Chart(combined_df).mark_line(color='
#aec7e8').encode(
268         x=alt.X('DateObject:T', title='日期'),
269         y=alt.Y('EmploymentRate:Q', title='就业率', scale=alt.
Scale(domain=[0, 1]), axis=alt.Axis(format='.0%')),
270         detail='Year:N'
271     )
272
273     points_actual = alt.Chart(df_actual).mark_point(
274         filled=True, size=60, color='#1f77b4'
275     ).encode(
276         x='DateObject:T',

```

```

277         y='EmploymentRate:Q',
278         tooltip=[
279             alt.Tooltip('DateObject:T', title='日期'),
280             alt.Tooltip('EmploymentRate:Q', title='实际就业率'
, format='.2%')
281         ]
282     )
283
284     chart = (line_historical + points_actual).properties(
285         title='问题三：高保真度数据插值结果',
286         width=800,
287         height=400
288     ).interactive()
289
290     chart.save('problem_three_chart.html')
291
292     print("问题三求解完成。")
293     print("插值结果已保存至 '附件四_插值结果.csv'")
294     print("可视化图表已保存至 'problem_three_chart.html'")
295
296
297 if __name__ == '__main__':
298     solve_problem_three()

```